

Università degli Studi di Napoli Federico II

Esame di Programmazione II

Simulazione Prova d'esame su Proxy-Skeleton

Durata della prova: 120 minuti

Cognome..... Nome..... Matr.....

Lo studente legga attentamente il testo e produca il programma ed i casi di test necessari per dimostrarne il funzionamento. La mancata compilazione completa dell'elaborato, la compilazione con errori o l'esecuzione errata dell'algoritmo daranno luogo alla valutazione come **prova non superata**. Ricordarsi di indicare Cognome, Nome e matricola su questo stesso foglio, che dovrà essere in ogni caso consegnato alla Commissione. Al termine della prova lo studente dovrà far verificare il funzionamento del programma ad un membro della Commissione.

Testo della prova

Il candidato realizzi un'applicazione server multithread per la prenotazione di appelli di esame. Un **server multithread** realizzato attraverso un **pool** di 5 thread accetta le richieste di prenotazione, ed implementa il servizio booleano `prenota(int matricola)`. A ogni richiesta, le matricole sono memorizzate in una coda circolare di dimensione pari a 10 che funge da buffer temporaneo. Un **thread dedicato** (implementato lato server, *StorageThread*) provvede a svuotare la coda ad intervalli regolari di 5 secondi e memorizza le matricole sul file `elenco-prenotati.txt`. L'accesso alla coda è gestito in mutua esclusione.

Il server deve fornire **2 modalità** di prenotazione: prenotazione *senza* e *con limitazione* al numero massimo di prenotati. L'operazione `prenota` restituisce esito `true` (prenotazione effettuata) o `false` (prenotazione non effettuata) come indicato in seguito. Nella modalità *senza limitazione*, ogni richiesta è messa in attesa fin quando si libera un posto nella coda: tutte le richieste di prenotazione terminano con esito `true`. Nel caso *con limitazione*, se la coda è piena, l'operazione termina immediatamente con esito `false` senza attendere ulteriormente.

L'**applicazione client** genera 10 thread. Ogni thread invia 5 richieste (le matricole inviate al server sono generate in maniera casuale) con frequenza di 1 richiesta al secondo.

La comunicazione tra client e server avviene per mezzo di **socket**. L'applicazione dovrà essere realizzata per mezzo del design pattern **proxy-skeleton**.

